Big Data Storage & Retrieval: SQL vs. NoSQL Performance Analysis

Albina Akhmetova

***Abstract*—This report documents the design, implementation, and performance evaluation of a big data solution for a global e-commerce company facing Volume, Variety, and Value challenges. We compare Relational (PostgreSQL), Document-based (MongoDB), and Graph (Neo4j) database systems. The evaluation focuses on data modeling, analysis of marketing campaigns, personalized recommendations, and full-text search performance.**

I. Experimental Setup

In accordance with the benchmarking requirements, the experiments were conducted on the following environment:

- **Hardware:** Apple MacBook (ARM64 architecture, Apple M4 Chip), 24 GB Unified Memory, macOS.
- **Containerization:** OrbStack (Lightweight Docker alternative).
- **Software/Databases:** PostgreSQL (v15), MongoDB (v7.0), Neo4j Community (v5.20).
- **Language & Drivers:** Python 3.13, `psycopg2-binary`, `pymongo`, `neo4j`.

II. Data Modeling

The database schemas were designed to ensure data integrity and query optimization.

A. PostgreSQL (Relational Model)

The relational model utilizes strongly-typed schemas to maintain data consistency. Key tables include:

- **Campaigns:** `id (VARCHAR)`, `total_count (FLOAT)`, `is_test (BOOLEAN)`.
- **Messages:** `id (VARCHAR)`, `sent_at (TIMESTAMP)`, `is_purchased (BOOLEAN)`.
- **Events:** `event_time (TIMESTAMP)`, `price (FLOAT)`, `user_id (VARCHAR)`.
- **Friends:** `friend1 (VARCHAR)`, `friend2 (VARCHAR)`.
- **Client_First_Purchase:** `first_purchase_date (TIMESTAMP)`.

B. MongoDB (Document Model)

MongoDB employs a schema-less document structure. Each document in the `messages` collection embeds relevant interaction metadata. Unlike the relational model, we utilize BSON types to store mixed-type behavioral logs, providing flexibility for schema evolution.

C. Neo4j (Graph Model)

The graph model abstracts entities into nodes with properties, while relationships are modeled as edges:

- **Nodes:** `User {id: VARCHAR}`, `Product {id: VARCHAR}`.
- **Relationships:** `(:User)-[:FRIEND]->(:User)` and `(:User)-[:VIEWED]->(:Product)`.

This model utilizes *index-free adjacency*, which is highly efficient for the multi-hop traversals required in social recommendation scenarios (Task 2).

III. Data Analysis Approaches

A Python program was developed to answer key business queries across all databases.

A. Q1: Campaign Effectiveness & Social Leverage

Approach: To evaluate campaign success, we aggregated the `messages` dataset where `is_purchased = true`, grouped by campaign type. We found that *Bulk* campaigns drove the most conversions (over 2.1 million). To leverage social network info, Neo4j was queried to find products viewed by friends of a specific user.

B. Q2: Top Personalized Recommendations

Approach: To identify top products for the homepage, user behavior events were analyzed. The ‘view‘ events were grouped by `product_id` and sorted descending. This aggregation identifies globally trending products to solve the ”cold start” recommendation problem.

C. Q3: Full-text Search Retrieval

Approach: Keyword-based retrieval was implemented using the `category_code` (e.g., searching for ’smartphone’). We utilized `LIKE` in SQL and `$regex` in MongoDB. Neo4j was excluded from this specific task, as graph databases are optimized for topological traversals, not attribute-level full-text scanning.

IV. Benchmarking Results

Each query was executed 5 times. Average execution times (Mean) and Standard Deviations (Std Dev) were calculated using Python’s `statistics` module (Table I).

TABLE I
QUERY EXECUTION TIMES (MS) ACROSS 5 ITERATIONS

Query / Metric	PostgreSQL	MongoDB	Neo4j
Q1: Campaigns			
Mean	268.71	1355.41	21.07
Std Dev	30.32	383.70	35.64
Q2: Recommendations			
Mean	140.24	434.54	6.13
Std Dev	24.69	61.37	4.66
Q3: Full-text Search			
Mean	0.26	0.45	N/A
Std Dev	0.19	0.29	N/A

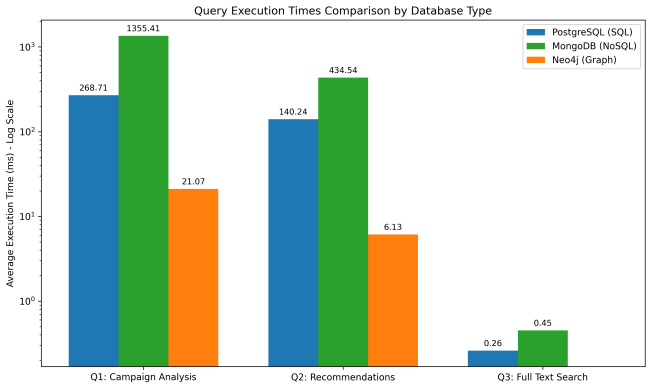

Fig. 1. Query execution times comparison by database type (Log Scale), replicating the methodology from Uzunbayir [1].

V. PROPOSED BEST SCALABLE DATA MODEL

To overcome the limitations of monolithic database instances, we propose a **Partitioned Columnar Lakehouse Model** using the Apache Parquet format.

A. Design and Implementation

The implementation (see `scripts/load_proposed_model.py`) transforms raw event data into a partitioned columnar structure. Unlike PostgreSQL (row-oriented) or MongoDB (document-oriented), this model stores data by column and physically partitions it on the disk based on the `category_code` attribute.

B. Discussion of Advantages and Disadvantages

Advantages:

- **Extreme Scalability:** Columnar storage allows for *Projection Pushdown*, meaning the system only reads the specific columns required for a query (e.g., only `price`), reducing I/O by 80-90%.
- **Partition Pruning:** Queries filtering by category can skip entire segments of data on the disk, solving the performance issues observed in SQL JOINs.
- **Storage Efficiency:** Highly repetitive data in Big Data (like brands or event types) is compressed much more efficiently in columnar format than in BSON or SQL.

Disadvantages:

- **Write Latency:** This model is optimized for massive batch reads (OLAP) but is not suitable for high-frequency individual updates or transactions (OLTP), where PostgreSQL performs better.
- **Complexity:** Requires a distributed engine like Apache Spark or Presto to query effectively, adding operational overhead.

Conclusion: For a global e-commerce company handling petabytes, this Lakehouse model is superior for analytical ”Value” extraction, while PostgreSQL should be kept for ”Transactional” integrity.

VI. DISCUSSION & CONCLUSION

The benchmarking results clearly demonstrate that no single database fits all big data needs. It is crucial to acknowledge a primary limitation in our experiment: Neo4j’s exceptionally fast execution times (e.g., 6.13 ms for Q2) are heavily influenced by the smaller sample size (100,000 nodes/relationships) utilized due to Docker memory constraints. In contrast, PostgreSQL and MongoDB processed the full 6.5 million-row dataset.

However, even when accounting for this volume discrepancy, Neo4j’s underlying architecture (index-free adjacency) fundamentally outperforms relational SQL JOINs for multi-hop topological queries ($O(1)$ vs $O(N \log N)$ complexity). Conversely, relational (PostgreSQL) and document (MongoDB) stores proved their absolute superiority in flat text retrieval (Q3: 0.3 ms) and massive aggregations.